Matrix Search — Recipe Search Specification

Overview

This document describes the recipe search requirements for Matrix Search. It defines the input data (raw Workato recipe JSON), the three categories of user queries the system must handle, the fields we currently plan to search on, the evaluation dataset, and the metrics used to measure retrieval quality. The searchable fields reflect the current plan — additional fields such as recipe tags may be added in the future.

The goal is to close the gap between the people who build recipes and the people who use them. This is the foundation for something much larger. Once the platform understands what automations do in business terms, the path opens to process-level navigation, cross-recipe dependency maps, and eventually operational metrics answered directly from the platform.

Background

The platform today knows only connectors, actions, and field names — it cannot answer "what handles our onboarding?". As automation estates grow, this creates shadow documentation and tribal knowledge: the automation layer becomes a black box only its builders can navigate.

Semantic recipe search closes that gap by letting users find automations by meaning — whether that is a business goal, a specific connector, or a field dependency. The dependency lookup use case is a direct response to change management needs: when a connector action is updated or deprecated, teams need to know every affected recipe. This is a top customer ask and a recurring request from the AIRO product hour.

No single retrieval strategy covers all query types — semantic search misses exact technical identifiers, while keyword matching fails entirely on business language. The three requirements below address each pattern.

Input Data

Each recipe is provided as a raw JSON string (the `pii_removed_code` column from `bt_prod.parquet`). The JSON represents a single recipe version with PII-redacted step content.

A recipe is a recursive step tree. Each step node contains:

JSON Field	Type	Description
`keyword`	string	Step type: `trigger`, `action`, `if`, `foreach`, `try`, `catch`
`provider`	string	Connector/app name, e.g. `salesforce`, `workato_db_table`, `slack_bot`
`name`	string	Operation name within the provider, e.g. `search_records`, `upsert_record`

JSON Field	Type	Description
`input`	object	Key-value pairs of configured input parameters (values PII-redacted)
`extended_input_schema`	array	Schema definition for inputs — contains parameter names and types
`extended_output_schema`	array	Schema definition for outputs — contains the datapill field names
`comment`	string	Optional free-text annotation by the recipe author (not redacted)
`block`	array	Child steps (recursive nesting for `if`, `foreach`, etc.)

Available (not redacted): `keyword`, `provider`, `name`, `extended_input_schema`, `extended_output_schema`, `comment`, and step nesting structure.

Redacted: Configured input values (URLs, IDs, SQL strings, data pill expressions) and block-level `title` / `description` fields.

Example

```
{
  "keyword": "trigger",
  "provider": "salesforce",
  "name": "updated_record",
  "input": { "object": "Contact", "since": "-- REDACTED --" },
  "block": [
    {
      "keyword": "action",
      "provider": "workato_db_table",
      "name": "upsert_record",
      "comment": "Write updated contact back to the internal DB.",
      "input": { "table_id": "-- REDACTED --", "record_id": "-- REDACTED --
" },
      "extended_output_schema": [
        { "name": "record", "type": "object" },
        { "name": "continuation_token", "type": "string" }
      ]
    },
    {
      "keyword": "action",
      "provider": "slack_bot",
      "name": "post_message",
      "input": { "channel": "-- REDACTED --", "text": "-- REDACTED --" }
    }
  ]
}
```

Requirements

REQ-1 — Recipe Indexing

What needs to be done

Index recipes from raw JSON into Matrix Search. Each recipe must be stored with its `flow_id` as the unique identifier and the following 4 extracted fields, derived by recursively walking the step tree:

Field	Extracted from	Examples
provider	`"provider"` field on every step (all unique values)	`salesforce`, `workato_db_table`, `slack_bot`, `google_sheets`
actions	`"provider"` + `"name"` on every step	`salesforce/search_records`, `workato_db_table/upsert_record`, `slack_bot/post_message`
input_fields	Input parameter key names from `input` and `extended_input_schema`	`table_id`, `record_id`, `continuation_token`, `object_type`
datapill_fields	Output field names from `extended_output_schema`	`record`, `records`, `continuation_token`, `package_version_name`

The following fields are under consideration for future indexing:

Field	Source	Purpose
action_description	Human-readable description of each action step (from connector metadata or step `comment`)	Improves matching on business-language queries by providing natural language context for each action
recipe_description	LLM-generated summary of the recipe's business purpose	Primary signal for semantic search (REQ-2); bridges the vocabulary gap between user queries and raw technical recipe content

Additional fields — such as recipe tags — may also be added as requirements evolve.

Expectations

- Each recipe is retrievable by its `flow_id`
- Records can be created, updated, and deleted individually (recipes are updated when authors publish new versions)
- Field extraction is applied at index time from the raw JSON

Limits

- Corpus size: ~10,000 recipes today; plan for up to ~1,000,000 at scale
- Each recipe JSON: up to ~500 KB

- Index build time: no hard requirement for now; throughput for batch re-indexing should be documented
- Availability: index must be queryable within 60 seconds of a recipe being created or updated

Timeline: Early Q2

REQ-2 — Semantic Search (Business Language Queries)

What needs to be done

Support retrieval of recipes using natural language queries that describe a business process or goal, with no technical vocabulary. The system must use semantic (dense vector) matching to bridge the vocabulary gap between the user's language and the recipe's technical content.

Example:

Query: *"How can we automatically create a support ticket and notify the team when a customer issue is submitted?"*

Retrieved recipe (`8621_45185597_v2`):

```
Recipe body:
  Connectors: fresh_desk, intercom, logger, slack_bot, stripe,
workato_service
  Steps:
  - trigger: workato_service / receive_request
    - action: stripe / get_customer_by_id
    - action: intercom / search_user
    - action: fresh_desk / __adhoc_http_action   # create support ticket
    - action: slack_bot / post_bot_message        # notify team

Indexed fields:
  provider:        fresh_desk, intercom, logger, slack_bot, stripe,
workato_service
  actions:         fresh_desk/__adhoc_http_action, intercom/search_user,
                   slack_bot/post_bot_message, stripe/get_customer_by_id,
...
  input_fields:    channel, email, id, message, mnemonic, name, ...
  datapill_fields: account_id, active, address_city, address_country, ...
```

Note: the query contains no connector names or technical terms — the match is purely on semantic meaning via the recipe body embedding.

Expectations

- Returns top-K results ranked by relevance, each containing the original recipe document and its relevance score
- No formatting or result transformation required — formatting and matching are handled on the agent/tool side
- MRR@5 ≥ 0.4 on the Category 1 evaluation dataset (50 queries)

Limits

- Top-K: 5 results per query
- Maximum query latency: TBD (to be agreed)
- Concurrency: TBD (to be agreed)

Timeline: Early Q2

REQ-3 — Technical Feature Search (Provider & Action Queries)

> The example below shows the user-facing query. How the agent translates it into a Matrix Search query is TBD — the requirement defines the retrieval capability, not the wire format.

What needs to be done

Support retrieval of recipes by specifying one or more apps, connectors, or operation types. Queries use technical vocabulary (connector names, action names) that appears literally in the recipe. Both exact token matching and semantic matching contribute.

Example:

Query: *"Which automation runs on a schedule to create Google Sheets pipeline reports by copying Google Drive spreadsheet templates, populating rows from a Google Sheet, and posting the report links to Slack?"*

Retrieved recipe (`50737_59325567_v8`):

```
Recipe body:
  Connectors: clock, google_drive, google_sheets, lookup_table, slack_bot,
              workato_db_table, workato_recipe_function, workato_smart_list
  Steps:
  - trigger: clock / scheduled_event
    - action: google_sheets / get_spreadsheet_rows_v4
    - foreach [loop]
      - action: google_drive / copy_file         # copy spreadsheet
template
      - action: google_sheets / __adhoc_http_action
      - action: workato_db_table / get_records
    - action: slack_bot / post_bot_message       # post report link

Indexed fields:
  provider:        clock, google_drive, google_sheets, lookup_table,
slack_bot,
                   workato_db_table, workato_recipe_function,
workato_smart_list
  actions:         clock/scheduled_event, google_drive/copy_file,
                   google_sheets/get_spreadsheet_rows_v4,
                   slack_bot/post_bot_message,
workato_db_table/get_records, ...
  input_fields:    blocks, channel, col_sep, filters, flow_id, headers, ...
```

```
    datapill_fields: col_account_id, col_account_name,
col_activating_opp_close_date, ...
```

Note: the query names `google_sheets`, `google_drive`, and `slack` — all present in the `provider` and `actions` indexed fields.

Expectations

- Returns top-K results ranked by relevance, each containing the original recipe document and its relevance score
- No formatting or result transformation required — formatting and matching are handled on the agent/tool side
- Matches on the `provider` and `actions` fields as the primary signal
- Tolerates minor typos in connector or action names
- Compound identifiers such as `workato_db_table` and `salesforce/search_records` must survive tokenization intact — tokenizers that split on underscores or slashes will degrade precision
- MRR@5 ≥ 0.8 on the Category 2 evaluation dataset (50 queries)

Limits

- Top-K: 5 results per query
- Maximum query latency: TBD (to be agreed)
- Concurrency: TBD (to be agreed)

Timeline: Early Q2

REQ-4 — Dependency & Field Lookup (Impact Analysis Queries)

> The example below shows the user-facing query. How the agent translates it into a Matrix Search query is TBD — the requirement defines the retrieval capability, not the wire format.

What needs to be done

Support retrieval of all recipes that reference a specific provider, action, input field, or datapill field — primarily for change management and impact analysis. When a connector is updated, deprecated, or modifies an action, users must be able to identify every affected recipe. This is one of the most commonly requested features from customers and a top ask following the AIRO product hour.

Example:

Query: *"If the workato_db_table get_records action changes (table_id / order_by_field_id / filters / continuation_token), which recipes will be affected?"*

Retrieved recipe (`136763_23098236_v2`):

```
Recipe body:
  Connectors: grammarly_scim__connector, workato_db_table,
workato_recipe_function
  Steps:
```

```
  - trigger: workato_recipe_function / execute
    - action: grammarly_scim__connector / search_records   # check if user
exists
    - action: grammarly_scim__connector / update_record    # activate user
    - action: grammarly_scim__connector / create_record    # create user
    - foreach [loop]
      - action: workato_db_table / get_records
          fields: table_id, filters, order_direction, limit
    - action: workato_recipe_function / return_result

Indexed fields:
  provider:        grammarly_scim__connector, workato_db_table,
workato_recipe_function
  actions:         grammarly_scim__connector/create_record,
                   grammarly_scim__connector/search_records,
                   grammarly_scim__connector/update_record,
                   workato_db_table/get_records,
                   workato_recipe_function/execute, ...
  input_fields:    active, count, email, filters, id, limit, object,
order_direction,
                   table_id, ...
  datapill_fields: active, app_access_level_id, continuation_token, email,
first_name,
                   id, ...
```

Note: the query names the exact action (`workato_db_table/get_records`) and input fields (`table_id`, `filters`, `continuation_token`) — all present in the `actions` and `input_fields` indexed fields. The match depends on those identifiers surviving tokenization intact.

Expectations

- Returns top-K results ranked by relevance, each containing the original recipe document and its relevance score
- No formatting or result transformation required — formatting and matching are handled on the agent/tool side
- Matches on the `actions`, `input_fields`, and `datapill_fields` structured fields as the primary signal
- Exact matching on compound identifiers (e.g. `continuation_token`, `workato_db_table`) is required — semantic similarity alone is insufficient for this category
- Results are exhaustive: all recipes referencing the specified field or action should be retrievable, not just a semantically similar subset
- MRR@5 ≥ 0.8 on the Category 3 evaluation dataset (50 queries)

Limits

- Top-K: 5 results per query
- Maximum query latency: TBD (to be agreed)
- Concurrency: TBD (to be agreed)

Timeline: Mid Q2

Evaluation Metrics

All metrics are computed at `k = 5` (top-5 results).

Metric	Definition
Recall@5	Fraction of ground-truth strong recipes retrieved in the top 5
MRR	Mean Reciprocal Rank — average of 1/rank of the first strong hit

Primary metric: **MRR** per requirement category.

Requirement	MRR@5 target
REQ-2 — Business Language	≥ 0.4
REQ-3 — Technical Feature	≥ 0.8
REQ-4 — Dependency Lookup	≥ 0.8